# Session 4

## 1.Use the online API testing tool ReqRes (https://reqres.in/) to make a GET request to /api/users/2 and note down the response status code and the returned user's first name.

### GET request — Retrieve user details

Objective: To retrieve the details of a user using the ReqRes API.

- Website: ReqRes API
- HTTP method: GET
- Endpoint: `/api/users/2`

Steps performed:

1. Open the ReqRes website or an API testing tool such as Postman.
2. Enter the endpoint `https://reqres.in/api/users/2`.
3. Select the GET method and send the request.
4. Check the response status code and the JSON response body.

Expected result (classic ReqRes example):


```
{
  "data": {
    "id": 2,
    "email": "janet.weaver@reqres.in",
    "first_name": "Janet",
    "last_name": "Weaver"
  }
}
```


- Expected status code: `200 OK`
- User's first name: `Janet`

Conclusion: The GET request retrieves the details of user ID 2 successfully if the API returns status code 200.

## 2. Send a POST request using ReqRes to /api/users with JSON data {"name": "Virat", "job": "Cricketer"} and write down the response code and the id value returned in the response.

# POST request — Create a new user

Objective: To send user details to the API and check whether a new user is created successfully.

- HTTP method: POST
- Endpoint: `https://reqres.in/api/users`
- Request body (JSON):

```
{
  "name": "Virat",
  "job": "Cricketer"
}
```

Steps performed:

1. Open ReqRes or Postman.
2. Enter the endpoint and select POST.
3. Set the `Content-Type` header to `application/json`.
4. Enter the JSON data in the request body.
5. Send the request and check the response.

Expected result (classic ReqRes example):

```
{
  "name": "Virat",
  "job": "Cricketer",
  "id": "123",
  "createdAt": "2026-10-20T10:00:00.000Z"
}
```

Note: This is an illustrative response. The actual ID and creation timestamp are generated by the API and may differ.

- Expected status code: `201 Created`
- Returned ID: Record the actual `id` value shown in your response.

Conclusion: The POST request is successful when the API returns status code 201 and includes the submitted user details and a generated ID.

Important: ReqRes has updated its API requirements. Current requests may require an API key in the `x-api-key` header. Check the official ReqRes documentation if your request fails. The status codes and response examples above describe the classic assignment endpoints.



ReqRes
+1

# 3. Given the following JSON response from an API: {"id": 101, "name": "Pizza Hut", "location": "Ahmedabad"}, write the SQL INSERT statement to add this restaurant to a table named restaurants with columns id, name, and location.

## SQL INSERT — Add a restaurant

Given API response:

```
{
  "id": 101,
  "name": "Pizza Hut",
  "location": "Ahmedabad"
}
```

SQL query:

```
INSERT INTO restaurants (id, name, location)
VALUES (101, 'Pizza Hut', 'Ahmedabad');
```

Explanation: The `INSERT INTO` statement adds a new record to the `restaurants` table. The values correspond to the restaurant ID, name, and location.

Expected result: A new row is added with ID `101`, name `Pizza Hut`, and location `Ahmedabad`, assuming the table exists and the ID does not violate a constraint.

# 4. Suppose you have a users table in your database and the UI shows a new user 'Riya' was registered. Write an SQL SELECT query to confirm if 'Riya' exists in the users table, and explain what result you would expect if the UI and DB are in sync.

## SQL SELECT — Verify whether Riya exists

Objective: To check whether a user named Riya is present in the `users` table after registration through the UI.

SQL query:

```
SELECT *
FROM users
WHERE name = 'Riya';
```

Explanation: The query searches the `users` table for records whose `name` is `Riya`.

Expected result:

- If the UI and database are in sync, the query should return Riya's record if the registration was saved successfully.
- If no record is returned, Riya may not have been saved, or there may be a delay or a mismatch between the UI and database.

Conclusion: This SELECT query helps verify whether the newly registered user is stored in the database.

# 5. After deleting a product from your shopping cart in the Flipkart UI, write an SQL query to verify that the product is no longer present in the cart_items table for your user.<br><br>Hint: Use SELECT to check for the product_id and your user_id in cart_items.

## SQL SELECT — Verify product deletion from the cart

Objective: To check whether a product has been removed from the `cart_items` table after deleting it from the Flipkart UI.

SQL query:

```
SELECT *
FROM cart_items
WHERE product_id = YOUR_PRODUCT_ID
  AND user_id = YOUR_USER_ID;
```

Explanation: The query checks whether the specified product still exists in the cart for the specified user. Replace `YOUR_PRODUCT_ID` and `YOUR_USER_ID` with the actual numeric IDs.

Expected result:

- If the product was deleted successfully and the database is in sync with the UI, the query should return zero rows.
- If a row is returned, the product is still present in the user's cart in the database.

Conclusion: The SELECT query helps confirm whether the product deletion in the UI is reflected in the database.

# Final summary

| Question | Expected answer |
| --- | --- |
| 1. GET user 2 | Status `200 OK`; first name `Janet` in the classic example |
| 2. POST new user | Status `201 Created`; record the generated ID from the actual response |
| 3. Insert restaurant | Use `INSERT INTO restaurants` |
| 4. Check Riya | Use `SELECT` with `WHERE name = 'Riya'` |
| 5. Check deleted product | Use `SELECT` with both `product_id` and `user_id`; expect zero rows |

Before submitting: For Questions 1 and 2, perform the actual requests and record the response values you see. The examples above are suitable for understanding the assignment, but they are not proof that those exact responses were returned during a live test. For Questions 3–5, the SQL statements are ready to use, subject to your database's actual column names.